

ANISH NAIR

A1

25070122037

Case Studies of Java using Leetcode (2+8)

The image displays two screenshots of the LeetCode IDE interface, showing solutions for two different problems in Java.

Top Screenshot: 55. Jump Game

Description: You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position. Return `true` if you can reach the last index, or `false` otherwise.

Example 1:
Input: `nums = [2,3,1,1,4]`
Output: `true`
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:
Input: `nums = [3,2,1,0,4]`
Output: `false`
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $0 \leq \text{nums}[i] \leq 10^5$

Code:

```
1 class Solution {
2     public boolean canJump(int[] nums) {
3         int maxReach = 0;
4         for (int i = 0; i < nums.length; i++) {
5             if (i > maxReach) {
6                 return false;
7             }
8             maxReach = Math.max(maxReach, i + nums[i]);
9         }
10        return true;
11    }
12 }
```

Testcase: Accepted Runtime: 0 ms

Case 1: Input: `nums = [2,3,1,1,4]` Output: `true`

Case 2: Input: `nums = [3,2,1,0,4]` Output: `false`

Bottom Screenshot: 3. Longest Substring Without Repeating Characters

Description: Given a string `s`, find the length of the **longest substring** without duplicate characters.

Example 1:
Input: `s = "abcabcbb"`
Output: `3`
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.

Example 2:
Input: `s = "bbbbb"`
Output: `1`
Explanation: The answer is "b", with the length of 1.

Example 3:
Input: `s = "pwwkew"`
Output: `3`
Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq \text{s.length} \leq 5 \times 10^4$

Code:

```
1 class Solution {
2     public int lengthOfLongestSubstring(String s) {
3         int maxlen = 0;
4         int n = s.length();
5
6         for (int i = 0; i < n; i++) {
7             boolean[] seen = new boolean[128];
8             for (int j = i; j < n; j++) {
9                 char c = s.charAt(j);
10                if (seen[c]) {
11                    break;
12                }
13                seen[c] = true;
14                int len = j - i + 1;
15                if (len > maxlen) maxlen = len;
16            }
17        }
18        return maxlen;
19    }
20 }
```

Testcase: Accepted Runtime: 0 ms

Case 1: Input: `s = "abcabcbb"` Output: `3`

Case 2: Input: `s = "bbbbb"` Output: `1`

Case 3: Input: `s = "pwwkew"` Output: `3`

Problem List

54. Spiral Matrix

Medium

Topics

Companies

Hint

Given an $m \times n$ matrix, return all elements of the matrix in spiral order.

Example 1:

1	→	2	→	3
4	→	5		↓
7	←	8	←	9

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Example 2:

1	→	2	→	3	→	4
---	---	---	---	---	---	---

17.7K 375 165 Online

Code

Java

Auto

4 class Solution {
5 public List<Integer> spiralOrder(int[][] matrix) {
6 List<Integer> result = new ArrayList<>();
7 int top = 0, bottom = matrix.length - 1;
8 int left = 0, right = matrix[0].length - 1;
9
10 while (top <= bottom && left <= right) {
11 for (int i = left; i <= right; i++) {
12 result.add(matrix[top][i]);
13 }
14 top++;
15
16 for (int i = top; i <= bottom; i++) {
17 result.add(matrix[i][right]);
18 }
19 right--;
20
21 if (top <= bottom) {
22 for (int i = right; i >= left; i--) {
23 result.add(matrix[bottom][i]);
24 }
25 bottom--;
26 }
27
28 if (left <= right) {

Saved

Ln 37, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Problem List

238. Product of Array Except Self

Medium

Topics

Companies

Hint

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a 32-bit integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: nums = [1,2,3,4]
Output: [24,12,8,6]

Example 2:

Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]

Constraints:

- $2 \leq \text{nums.length} \leq 10^5$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is **guaranteed** to fit in a 32-bit integer.

26.3K 632 360 Online

Code

Java

Auto

1 class Solution {
2 public int[] productExceptSelf(int[] nums) {
3 int[] result = new int[nums.length];
4
5 for (int i = 0; i < nums.length; i++) {
6 int product = 1;
7 for (int j = 0; j < nums.length; j++) {
8 if (i != j) {
9 product = product * nums[j];
10 }
11 }
12 result[i] = product;
13 }
14 return result;
15 }
16 }

Saved

Ln 9, Col 49

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

nums =
[1,2,3,4]

Output

[24,12,8,6]

Problem List<>🔍

RunCtrl

Submit📄🔗

🔧🔗🔗🔗🔗🔗Premium

DescriptionEditorialSolutionsSubmissions

14. Longest Common Prefix

EasyTopicsCompanies

Write a function to find the longest common prefix string amongst an array of strings.
If there is no common prefix, return an empty string "".

Example 1:
Input: strs = ["flower","flow","flight"]
Output: "fl"

Example 2:
Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.

Constraints:

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] consists of only lowercase English letters if it is non-empty.

👍 21.6K🗨️ 775🌟🔗🕒389 Online

Code

JavaAuto

1class Solution {
2public String longestCommonPrefix(String[] strs) {
3String prefix = strs[0];
4
5for (int i = 1; i < strs.length; i++) {
6while (strs[i].indexOf(prefix) != 0) {
7prefix = prefix.substring(0, prefix.length() - 1);
8}
9}
10return prefix;
11}
12}

SavedLn 12, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2

Input
strs =
["flower","flow","flight"]

Output

Problem List<>🔍

Submit📄🔗

🔧🔗🔗🔗🔗🔗Premium

DescriptionEditorialSolutionsSubmissions

1. Two Sum

EasyTopicsCompaniesHint

Given an array of integers *nums* and an integer *target*, return indices of the two numbers such that they add up to *target*.
You may assume that each input would have **exactly one solution**, and you may not use the same element twice.
You can return the answer in any order.

Example 1:
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].

Example 2:
Input: nums = [3,2,4], target = 6
Output: [1,2]

Example 3:
Input: nums = [3,3], target = 6
Output: [0,1]

👍 69.4K🗨️ 2K🌟🔗🕒1890 Online

Code

JavaAuto

1class Solution {
2public int[] twoSum(int[] nums, int target) {
3int[] result = new int[2];
4result[0] = -1;
5result[1] = -1;
6
7for (int i = 0; i < nums.length; i++) {
8for (int j = i + 1; j < nums.length; j++) {
9if (nums[i] + nums[j] == target) {
10result[0] = i;
11result[1] = j;
12}
13}
14}
15return result;
16}
17}

SavedLn 17, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input
nums =
[2,7,11,15]

target =

Problem List < > 🔍

Submit

0

Premium

DescriptionEditorialSolutionsSubmissions

11. Container With Most Water

MediumTopicsCompaniesHint

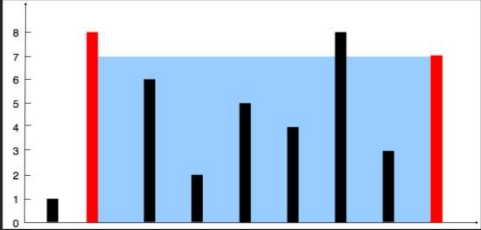
You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `ith` line are `(i, 0)` and `(i, height[i])`.

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:



34.7K 932 408 Online

Code

JavaAuto

```
1 class Solution {
2     public int maxArea(int[] height) {
3         int left = 0;
4         int right = height.length - 1;
5         int maxWater = 0;
6
7         while (left < right) {
8             int currentWater = Math.min(height[left], height[right]) * (right - left);
9             maxWater = Math.max(maxWater, currentWater);
10            if (height[left] < height[right]) {
11                left++;
12            } else {
13                right--;
14            }
15        }
16
17        return maxWater;
18    }
19 }
```

SavedLn 9, Col 57

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1

Case 2

Input

height =
[1, 8, 6, 2, 5, 4, 8, 3, 7]